WORKING WITH DOCKER IMAGES AND DOCKERHUB/ECR

CREATING TOMCAT CONTAINER

1. docker run –d -p 8080:8080 –name tomcat –rm tomcat
2. docker ps
3. <public_ip:8080>

Resolve tomcat error page

4. docker exec -it tomcat /bin/bash
5. cd /usr/local/tomcat/
6. mv webapps.dist/* webapps/
7. <public_IP:8080>

CREATE A HTTPD CONTAINER

1. docker run -d -P –name httpd –rm httpd
2. docker ps
3. docker inspect httpd
4. <public_ip:32768>
   {add the random IP in the security group}
5. <public_IP:32768>

To change the default httpd page

6. vim index.html
   Welcome to apache
7. docker cp index.html httpd:/usr/local/apache2/htdocs/index.html
8. <public_ip:32768>

DOCKER IMAGES

Docker image is a template that have all the dependencies and the configuration required to run the container.

1. docker pull httpd
2. docker images
3. docker run -d httpd

DOCKERHUB

Docker Hub is a cloud-based repository used to store, share, and download Docker images.

HOW TO CREATE A IMAGE FROM CONTAINER & STORE IT ON DOCKERHUB

1. docker run -d -p 8081:80 –name nginx –rm nginx
2. docker ps

create a image from container

3. docker commit nginx nginx:v1
4. docker images

Tag image with the dockerhub username

5. docker tag nginx:v1 mayurpandit25/nginx:v1

login with dockerhub

6. docker login -u username
   Username: <dockerhub_username>
   Password: <dockerhub_password>

Push image to dockerhub

7. docker push mayurpandit25/nginx:v1

Pull image from dockerhub

8. docker pull mayurpandit25/nginx:v1

ECR (ELASTIC CONTAINER REGISTRY)

1. it is a private Docker Hub for your applications
2. Used with services like Kubernetes or AWS ECS
3. Helps developers securely store container images on amazon ECR

CREATE A ECR REPOSITORY AND PUSH DOCKER IMAGES

1.create a instance

1.name ----------> docker
2.machine ------> ubuntu
3.key_pair ------> ubuntu
4.create a instance

2.access the instance

1.apt update
2.apt install docker -y

3.systemctl start docker

3.create a access key & secret key

4.access the instance

1.install aws cli

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

2.aws configure

Access_key:
Secret_key:

5.create a repository on ECR

1.go to ECR
2.click on repository -----> create a repository -----> frontend
3.view push commands ---> copy

5.Access the instance

1.paste these commands here

1. aws ecr-public get-login-password --region us-east-1 | docker login --username AWS --password-stdin public.ecr.aws/y8w2m7k8
2. docker tag nginx:latest public.ecr.aws/y8w2m7k8/frontend:latest
3. docker push public.ecr.aws/y8w2m7k8/frontend:latest

DANGLING IMAGES

1. Dangling images are unused Docker images without a tag.
    1. consume storage space
    2. That are not useful anymore
2. Images that are not associated with any container are called dangling images.

ALPINE IMAGES

1. Alpine Linux is a very small and lightweight Linux distribution used in Docker images.
2. These images uses the package manager apk to download the package

Features

1. Very small in size
2. Fast to download
3. Use less storage
4. More secure (less packages)

5. Faster container startup

we use alpine images for:

1. Node.js containers
2. Python apps
3. Nginx containers
4. Microservices
5. Kubernetes workloads

Example:

1. when u install ubuntu images this image having the size 70+mb
2. While when installing alpine image these images having size 5+mb

---

REMOVE DOCKER IMAGES

1. docker rmi <image_id> ----------> to remove a image
2. docker rmi -f <image_id> ----------> to remove image forcefully
3. docker image prune ----------> to remove dangling images <none>
4. docker image prune -a ----------> to remove all unused images
5. docker rmi $(docker images -q) ----> to remove all the images

---

HOW TO TAKE BACKUP OF CONTAINER

1. Create and run container
   docker run -d -p 8081:80 --name nginx_container nginx
2. Check image
   docker images
3. Save image backup into tar file
   docker save -o nginx_backup.tar nginx
4. Verify backup file
   ls
5. Restore image from backup
   docker load -i nginx_backup.tar
